Just build the tools yourself

You can have anything you want with so little effort.

JAMES STANIER

FEB 20, 2026 • PAID

5 1 Share

Welcome to February's subscribers edition, and thanks as always for your suppor This one's a bit different: less theory, more show and tell.

Here's why: AI has made building simple tools almost as easy as building a spreadsheet, and I'm going to walk you through some I've built over recent mont solve common problems without needing to buy specialist software.

By the end, I hope that you'll feel motivated to get your hands dirty and close to t details and solve real problems (and it's fun!).

Think about how internal tooling used to work. You had two options: convince engineering to build it for you, which meant competing for roadmap space again customer-facing features, or settle for off-the-shelf SaaS that didn't quite fit your workflow and required convincing the company to pay for it.

Good internal tooling was a luxury reserved for the biggest tech companies; the o who could afford dedicated teams to build custom software for management task everyone else, they made do with spreadsheets and workarounds.

Here's what we'll cover:

- Why the landscape has shifted and what that means for managers who used code.
- Four tools I've built recently, from a bragdoc generator to a planning tool th embodies the single prioritised list philosophy from this month's free article

- Some ideas for cool tools you could build.
- The practicalities: what this requires, where it breaks down, and when a tool served its purpose.

If you'd like to dig deeper into related themes, here are some articles from the ar

- Being in the details explores why staying close to what your teams are buildi matters more than ever.
- Use it or lose it covers the "minimum effective dose" of coding for managers want to keep their skills sharp.
- Should managers still code? tackles the fundamental question that this artic builds upon.

The new economics of tooling

The good news is that internal tooling is no longer a luxury reserved for large companies.

AI has democratised the ability to build custom tools, especially if you are time p and a little rusty. The economics have fundamentally changed: if you have some technical background, and even if it's been years since you wrote code professior you can now build a working tool in an afternoon.

The capabilities of coding assistants at the time of writing are good enough to sc countless common problems, even if what you need is something you'd have considered paying for just a few years ago.

This isn't about you as a leader pivoting back to individual contributor again, or putting yourself on the critical path for shipping features. It's about removing th friction between "I wish I had a tool that did X" and actually having it. The prob worth solving this way are the ones too niche for anyone else to build for you: yo specific workflow, your team's specific needs, the visibility gap that only matters your context.

So, let me show you what I mean. Recently, I've built a handful of tools that have become part of how I work. Few of them took more than an afternoon. All of the solve problems that would have sat unsolved before.

And the liberating thing, other than having useful tools that solve real problems, that knowing they don't need to be production-grade makes the process creative and something that just feels really good.

Cool things I've built

Bragdoc generator

I mentioned this one briefly in Use it or lose it, but it's worth expanding on here. Here's the context: performance review season is tedious: you sit down to write y self-assessment and realise you've forgotten half of what you actually did. The evidence is scattered across Linear tickets, GitHub PRs, Slack threads, and your increasingly unreliable memory.

The concept of a "brag document" comes from Julia Evans: a running record of y accomplishments that you update throughout the year so you're not scrambling t remember everything at review time. It's a brilliant idea, but I never managed to one up to date. So I built a tool to generate one for me instead.

My workplace uses Linear for everything, and I also track my personal work todc it, which makes it a comprehensive record of my work. Given that it's a system o truth, it follows that the actions I take in Linear represent my main achievement the week.

The bragdoc generator pulls issue descriptions and comments from Linear for th week, then uses a local LLM via Ollama to summarise what I've been working or it weekly to generate a document that tells me what I've been up to, which I can reflect on, use to plan the next week, and revisit at performance review time. It t about an hour to build with Claude Code.

It's not perfect, but it solves my problem well enough, and that's the point. Perfe the enemy of done. If you're interested in giving it a go, you can fork it yourself, repoint it to other tools if you don't use Linear.

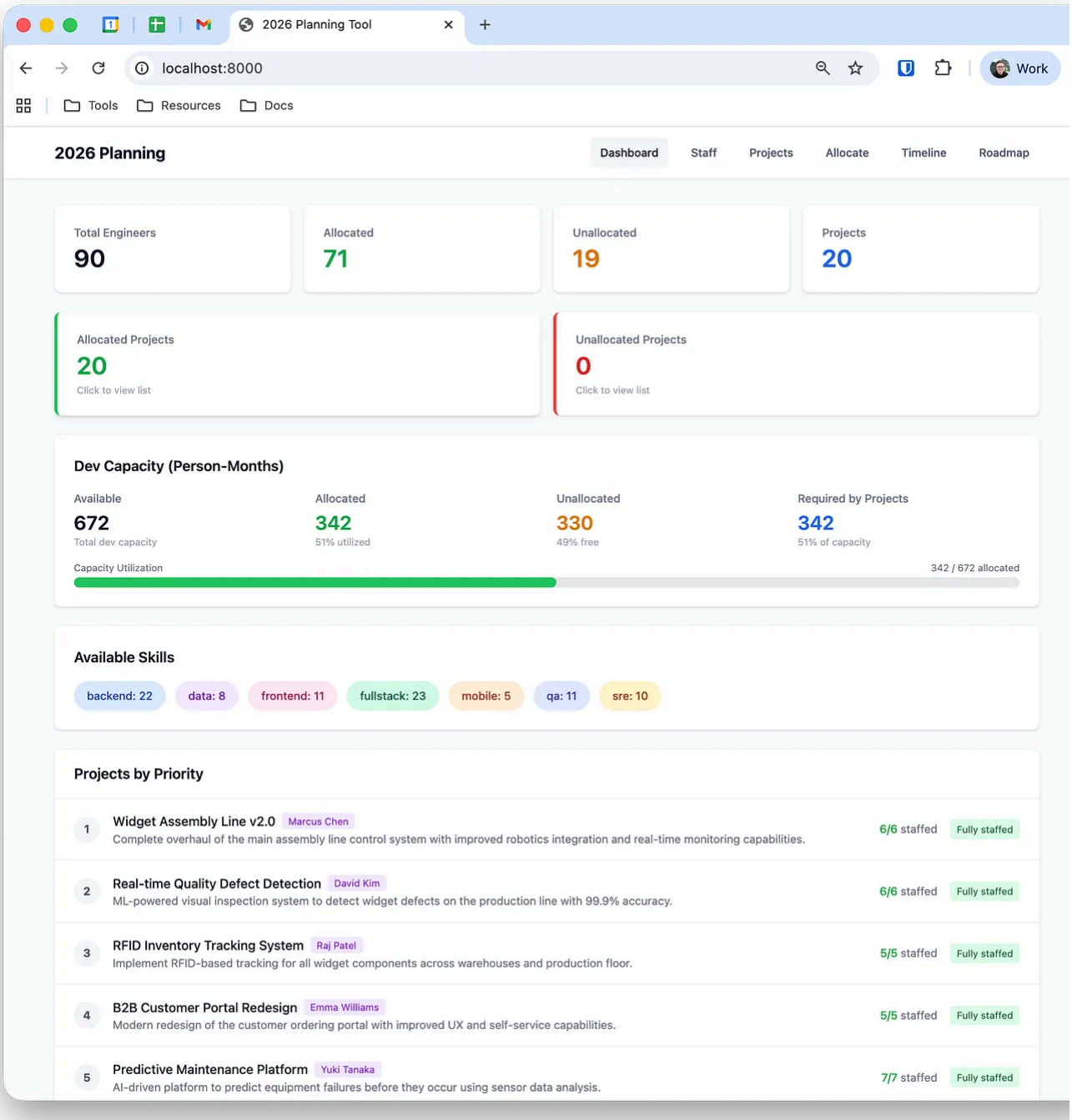
The planning tool

This month's free article introduced the single prioritised list: one ordered list of matters most, rather than the usual mess of backlogs, roadmaps, and priority tier scattered across different tools and documents. We wanted to use it to facilitate planning discussions at a high level for 2026, so I built something.

The requirements were more ambitious than the bragdoc:

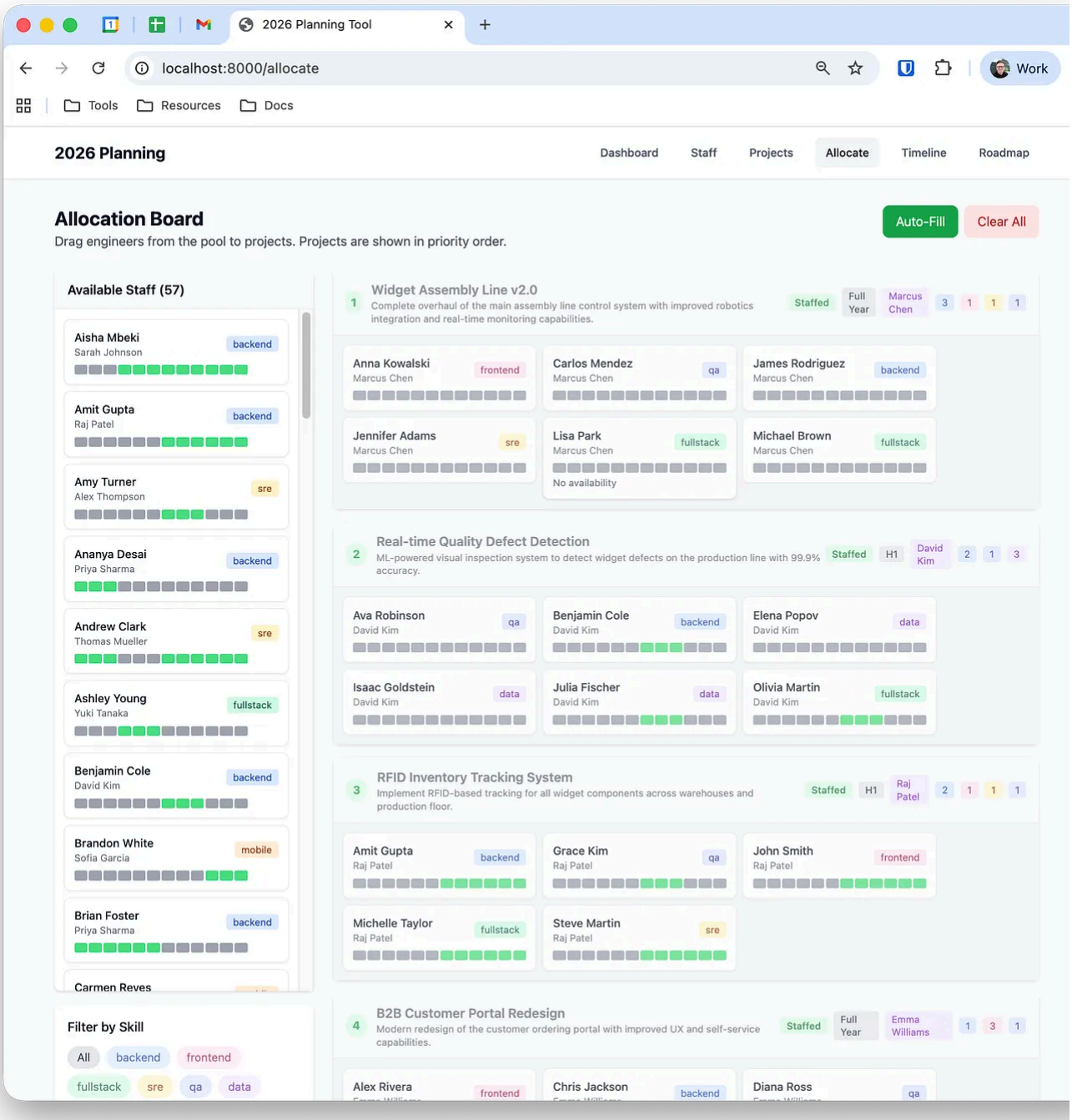
- A list of projects with their estimates and resourcing needs as submitted by teams, along with metadata like home team and required skills: backend, fro QA.
- A list of all staff with their skill sets and home teams.
- The ability to drag projects into priority order to create the single prioritised
- Manual allocation by dragging people onto projects.
- Auto-allocation that considers priority, available skills, and a preference for keeping engineers in their home teams.
- A roadmap view showing what gets fully staffed, what's partially resourced, what gets nothing at all.
- Import and export from CSV so we could pull data from existing spreadshee

Here's the first screenshot of what it looks like. All staff and projects are fictiona



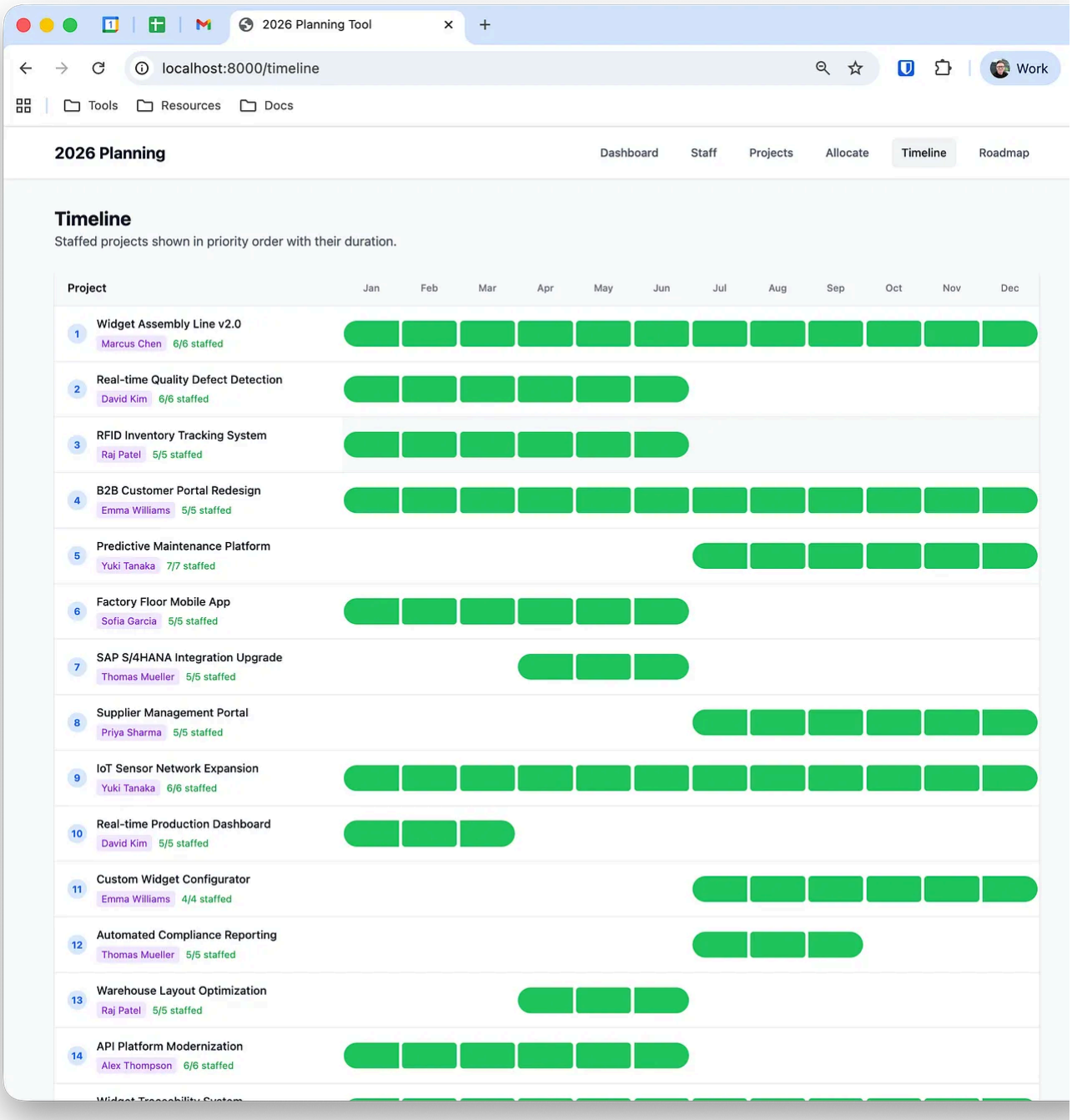
We started using it and iterated on it live, adding functionality as we went: bette colours to show what had and hadn't been prioritised, helper buttons for moving to the top and bottom of the list, and the ability to group items around vertical dividers.

Claude Code was building new features while we were using the tool, which mea could adapt it on the fly as needs emerged.



The tool made the conversation engaging and interactive: instead of staring at a spreadsheet, we could drag things around, run different scenarios, and build the features we wanted almost immediately and in the moment.

We also hosted it internally behind SSO so that we could collaborate asynchrono between sessions, which meant the planning conversation didn't have to stop wh meeting ended.



R&D Wrapped

If you've ever used Spotify, you'd have seen that at the end of the year they produ
Wrapped: a personalised summary showing your top artists, top tracks, and liste
habits across the year, including the categories and genres you've explored. Man
other pieces of software now follow this model.

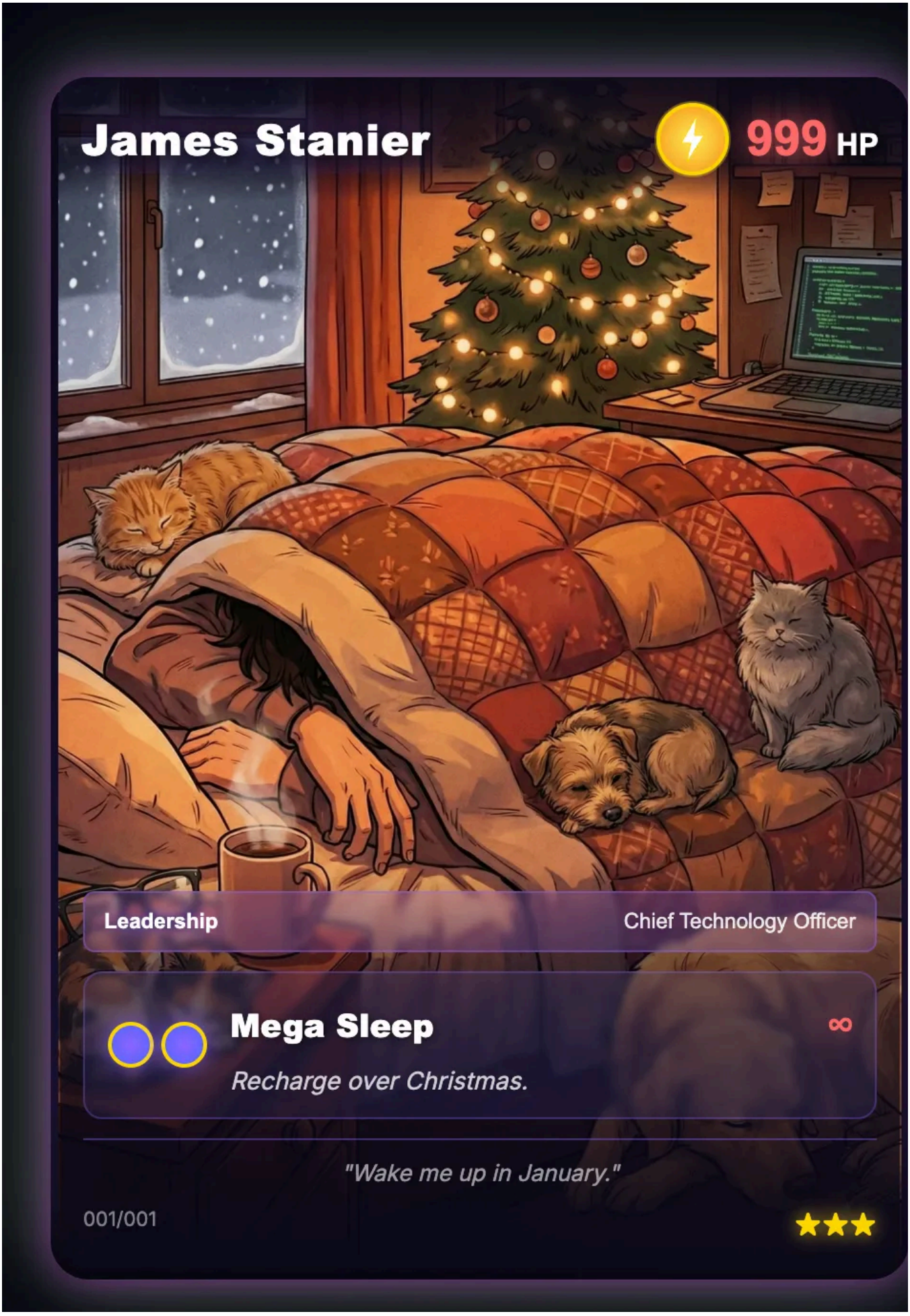
I'd seen previous companies build their own versions of Wrapped for their teams I wanted to create one for our department as a gift at the end of 2025.

But I also wanted to set myself a challenge: could I build the entire thing using o voice dictation and AI generation, no typing code, no manual edits, just Wispr Fl for speech-to-text, Claude Code for implementation, and a browser refresh to se results?

What started as a weekend project became something much bigger. I'd planned t spend a Saturday afternoon on it, but I ended up working across two weekends a few evenings because I was having so much fun.

Unrelatedly, I'd been watching videos online about tradeable cards and the mark behind them, Pokemon cards in particular, and I wanted that to be the core of th experience: collectible cards that people could discover and keep.


Contents
1 / 28
RESET
R&D Wrapped
2025
The Year that AI Arrived.
CONTROLS
Arrow Keys
PGUP
PGDN
Page Up/Down
A
Launch Animal
BINDER 26/29


James Stanier
999 HP
Leadership
Chief Technology Officer
Mega Sleep
∞
Recharge over Christmas.
"Wake me up in January."
001/001

I used Google's Nano Banana Pro to generate 29 collectible cards: highlights of t we shipped during the year, one for each of our teams, some representing interes cultural moments, and a handful of secret cards that you could only unlock by fir easter eggs or completing a hidden game.

The game itself was a side-scrolling shooter in the style of R-Type, a tongue-in-c representation of us trying to capture the total addressable market with our spac shooting down competitors.

It opened in a new tab, with the collection binder persisting across tabs via `localStorage` so that any cards you unlocked would appear in your collection regardless of where you found them.

This is where I got completely nerdsniped. I wanted the cards to be rotatable in 3 with shiny holofoil effects that had a light source emitting from the mouse point

I spent hours researching how to produce these effects online, then fed the exam into Claude Code to reproduce something similar. I even created different holofo styles for different card types: rainbow effects for rare cards, sparkles for others. of this was necessary, but it was enormously fun.

Building something this complex with only auto-generation taught me the impo of *layering*. I started with a wireframe page with no styling, then layered on the vi design, then built out one card and its interactions.

Once Claude Code understood which code could be reused, I could propagate pa across the site without duplication.

Working from the ground up kept the codebase manageable and meant I could k adding features without things falling apart or continually introducing bugs. Thi experience really showed me how you can do complex and in-depth things with a auto-generated code, but you do need to have an idea as to how to build software modular fashion in order to do so.

So did it go down well? I'd spent so much time on this that I was worried it woul flat when I shared it. However, the reception was better than I'd hoped: people genuinely enjoyed exploring the site, playing the game, and collecting all the car

If I did it again, I'd spend even *more* time on it, simply because I had so much fun building it. I'd also expand the scope: this year it was just for our R&D departme next time I'd love to do something for the whole company (although I'd need mo time for getting data and input).

Even though R&D Wrapped wasn't a tool in the same way as the previous two examples, it's worth reflecting on what it represents.

This kind of internal development, effectively a throwaway gift, would previously taken a whole team of engineers several weeks to put together. I was able to do it entirely on my own within just a few days.

As a leader, it's worth thinking about whether there are fun things you can do wi tooling that's now available, things that spread some cheer and happiness as wel just making yourself more efficient.

Claude Code as daily driver

The previous examples were standalone projects: tools built for a specific purpos then used. But the most significant shift in how I work hasn't been any single too been using Claude Code as my general unified interface for everything; an even b "bicycle for the mind" to borrow Steve Jobs' phrase.

The setup is straightforward: I have different project folders for different types o work, each with its own `CLAUDE.md` file that governs what kind of work I'm doi Claude Code in that particular environment.

One folder is for writing internal documents, pointed at my repository of articles help with proofreading and research, and loaded with my writing tone and style. Another is for working on KPIs and OKRs. My main workspace is what I call my driver": a folder connected to Linear, Notion, and our monitoring tools, with a `CLAUDE.md` that positions Claude as my executive assistant. Each folder become tailored workspace where Claude already knows what I'm trying to do and how I to do it.

On top of this, I connect Claude Code to my work tools via MCP. These integrati are installed as plugins: I have Linear, Notion, BetterStack, and Octopus Deploy connected, which means I can interact with all of them through conversation.

The most powerful part of my daily driver isn't any single command: it's the role system embedded in the `CLAUDE.md`. I've defined ten distinct personas, each wi specific triggers and behaviours. The default is Executive Assistant, which handl task lists, quick questions, and ticket creation.

But when I start a message with "what am I missing" or "poke holes in this", it switches to Sparring Partner mode and becomes explicitly adversarial, finding

weaknesses I'd otherwise miss. When I'm working on feedback for a direct repor activates Coach mode and structures everything around situation-behaviour-imp

The persona isn't cosmetic; it genuinely changes how Claude reasons about the problem. I've found myself defaulting to certain phrasings just to activate the rig mode, which means I'm thinking more deliberately about what kind of help I act need.

Here's what a real morning looks like. I ask "what's my current status in Linear? get back a categorised summary: what's in progress, what's blocked, what's due t No clicking through filters, no context switching.

Then a colleague sends a wall of Slack messages about an upcoming offsite: hote address, meeting times, dinner reservations, weather forecast, cab sharing arrangements. I paste the whole thing and say "create a travel prep ticket with su issues for flight check-in, train tickets, and packing." Claude extracts the key information, creates a parent issue with a structured itinerary in the description, adds the sub-tasks. The raw Slack dump becomes an organised reference I can cl from anywhere.

As I work through the tasks, I update them conversationally: "mark check-in as c set packing to in progress." When I realise I need to push a deadline, I say "move to next Friday" and it understands from context which ticket I mean.

The assistant also handles the small research tasks that would otherwise break m flow. Preparing for that trip, I needed to know if my backpack would fit Air Fran cabin baggage limits. I asked, it searched for the bag's dimensions, compared the against the airline's requirements, and gave me a yes or no. All without leaving tl conversation where I was managing my todo list.

I've also built slash commands for things I do repeatedly: `/weekly-ops` generat operations report by pulling data from BetterStack and Linear, `/dora-report` summarises our deployment metrics from Octopus Deploy, and `/check-sla` tel

if any bug fixes are at risk of breaching our SLA targets. I've borrowed liberally f community collections of slash commands too.

I also have a simple file called `inbox.md` that acts as a quick capture system. Throughout the day, I dump half-formed thoughts, Slack snippets, and notes fro conversations into it without worrying about structure. Then, when I have a mor run `/triage`. Claude reads the inbox, categorises each item as actionable, refere or done, and creates Linear tickets for anything with a clear next step. The raw n get converted into properly structured tasks with descriptions, and the inbox get cleared. It's a small thing, but it means I never lose track of something just becau didn't have time to process it properly in the moment.

The result is that my day often happens inside a single terminal window: I dump thoughts, reason through problems, take notes, create tasks, and occasionally spi small scripts and prototypes as I think.

This is the part that's hardest to convey until you've tried it: it's not about any sir tool or integration, but about how all of it compounds.

The workspace itself evolves too. Periodically I'll ask Claude to review the setup documentation and suggest what's out of date. It compares what's written agains what actually exists, flags the gaps, and updates the files. The system maintains i

The daily driver also helps with the parts of management I find hardest. I have a `performance/` directory with notes on each of my direct reports: observations throughout the year, peer feedback, and draft reviews. When performance review season arrives, I can ask Claude to synthesise everything into a first draft, structu around our company's review format.

More useful is the real-time help during the drafting process: I'll describe a situa and ask "how do I phrase this constructively?" and get back options that are hon but land well. It's like having a thought partner for the conversations that matter one that remembers everything I've noted down and helps me be more thoughtfu I'd manage on my own.

The tools you build feed back into your Claude setup, creating a personalised environment that reflects how you think and work. Instead of switching betweer applications, everything flows through one interface.

It's less *AI assistant* and more *externalised cognition*.

I use Claude Code, but similar workflows are possible with Cursor, GitHub Copi Windsurf, and others. The specific tool matters less than the approach.

What you could build

The examples above are things I've actually built and used. But part of the fun of new landscape is imagining what's possible. You know exactly what needs autom in your daily workflow, and you likely have a ton of ideas in your brain waiting tc out. Here are some to get you started.

1:1 prep generator. Before each 1:1 with a report, you want context so you can ha useful conversation: what have they been working on, what might be blocking th what did you discuss last time? A simple tool could pull together their recent contributions, current tickets, and the notes from your previous meeting. Instead spending ten minutes clicking through tabs to jog your memory, you get a single briefing document generated before each conversation, so you can show up prep and focused on them.

Team health dashboard. You probably already have access to engineering metric through whatever platform your company uses, but the dashboards are rarely configured for what you actually care about. A custom dashboard could surface e the signals that matter to your team: PR cycle time, deployment frequency, how l reviews sit waiting, the age of open bugs. Pull from your existing APIs, display it you want, and skip the enterprise pricing.

Stand-up generator. Every morning you need to remember what you did yesterda what you're doing today. A script that pulls your commits, merged PRs, and com tickets from the past 24 hours could generate a first draft of your stand-up updat

automatically. You edit it, add context, and you're done. Load it via a slash comm and it becomes part of your morning routine. No more staring at a blank text bo trying to recall what you spent Tuesday afternoon on.

Documentation generator. Nobody likes writing documentation, and it's always date. A tool that reads your codebase and generates a first draft of technical docs that turns incident resolution threads into runbooks, or that summarises a meeti transcript into shareable notes, removes the blank page problem. You still need t and refine, but starting from something is infinitely easier than starting from no Generating documentation is also a fantastic way of exploring a codebase for the time: prompt AI to explain how it works, ask for diagrams, and you'll understand faster than reading the code alone.

Feature flag cleanup tracker. Feature flags accumulate. Nobody knows which ar to remove. A dashboard showing flag name, age, last toggled date, owner, and wh it's still being evaluated in production makes the invisible visible. The cleanup w never feels urgent enough to prioritise, but a tool that surfaces the debt makes it harder to ignore.

Personal todo system. Every todo app is designed for someone else's brain. None them quite fit how you think. Build exactly the system that matches your mental model: your categories, your views, your rules for what surfaces when. It doesn't to work for anyone else, which is precisely what makes it worth building.

The tools that matter most are the ones only you would think to build. They're to niche for a startup to pursue, too specific to your context for off-the-shelf softwa solve. That's exactly what makes them worth an afternoon of your time.

The practicalities

Before we wrap, let me be honest about what this requires and where it breaks d

You need some technical background. You're reading this newsletter, so you pro already have it. But it's worth saying: this approach isn't (yet) for non-technical

managers.

If you used to code, even years ago, the rust comes off fast: the AI handles the sy while you handle the intent.

You don't need to remember how to configure webpack or write SQL from scrat you just need to know enough to describe what you want and recognise when something's wrong.

The best results come from treating it like a conversation with a lead engineer: a the role of product manager and user, say what you like and don't like, describe h you want it to work, and ask questions before diving in. Use planning mode to re what it's going to do before it does it. Anthropic's best practices guide is worth a

80% is often enough. These tools don't need to be production-grade. They don't 100% test coverage. They don't need to scale. They need to solve your problem w *enough* to be useful. "Good enough" is genuinely good enough. Resist the urge to polish: think MVP.

Maintenance debt is real, but manageable. These things bitrot: dependencies b APIs change, and something that worked fine six months ago suddenly doesn't. these tools are small enough to rewrite from scratch when they break, which cha the calculus entirely. For personal tools, rebuilding in an afternoon is often easie debugging, and sometimes the right call is simply to put it in the bin and start ag For tools others depend on, it's more complicated, which brings us to the next p

Know when to hand off. If a tool you built becomes load-bearing, if other people on it or it becomes critical to a process, it probably needs to become "real" softw That means discussing real support for it, documenting it properly, or at least ha an honest conversation about what happens if it breaks while you're on holiday.

Your turn

So: what's a problem you've been waiting for someone else to solve? What would build if it only took an afternoon? What manual process do you repeat every wee

could be automated?

The tools that matter most are the ones only *you* would think to build, because o *you* have your specific problems, your specific workflows, and your specific way thinking. Start small, pick something annoying, spend a Saturday afternoon on it see what happens.

If you build something, I'd love to hear about it! Drop me a message in the subsc chat.

Until next time.

5 Likes · 1 Restack

Discussion about this post

Comments Restacks

Write a comment...